Artificial Intelligence

Assignment-2

Report

Abdelrhman Awnar	8733
Taha Mostafa Mohamed	9539
Ahmed Tamer Abassy	8842

1. Introduction

Connect 4 is a two-player game where players alternately drop discs into a vertical grid, aiming to form the greatest number of four-in-a-row connections horizontally, vertically, or diagonally.

In this project, a **Human vs AI** version of Connect 4 was implemented to test three decision-making algorithms:

1. **Minimax**: basic adversarial search.
2. **Alpha-Beta Pruning**: optimized Minimax that skips unneeded branches.
3. **Expectimax**: uses weighted outcomes for possible opponent moves.

Since the full game tree is extremely large, each algorithm evaluates moves up to a fixed depth K using a heuristic function that measures how favorable a board is for the AI. The main goals are to compare these algorithms in performance and behavior, visualize their decision trees, and provide a simple GUI to play against them.

2. Game Implementation Overview

The program was developed in **Python** using **Tkinter** for the GUI and consists of two main files:

- **connect4.py**: manages game logic and AI algorithms.
    - Maintains the board as a 2-D list.
    - Provides move handling, scoring, and heuristic evaluation.
    - Implements Minimax, Alpha-Beta, and Expectimax with tree output for visualization.
- **gui.py** — handles user interaction and visualization.
    - Displays the 7 × 6 board with colored discs and simple animations.
    - Lets the player choose who starts and which algorithm to use.

- Shows the AI's decision tree after each move in a readable text panel.

Together, these create a full interactive game where the user can play Connect 4 against different AI strategies while observing their search logic.

3. AI Algorithms

The project implements three different search algorithms for the AI opponent, each providing a distinct way of exploring the game tree and selecting the best move.

Minimax

Minimax is the baseline adversarial search algorithm used to decide the optimal move for the AI player. It simulates all possible moves by both players up to a fixed search depth K. At each level, the AI selects the move that maximizes the heuristic value, while the human opponent is assumed to choose moves that minimize it.
This version evaluates every branch of the game tree, which makes it conceptually simple but computationally expensive at higher depths.

```
if maximizing_player:
    max_eval = float('-inf')
    for col in range(self.width):
        row = self.simulate_drop(col, 1)
        if row is None: # dont calculate for invalid m
            continue
        eval , subtree = self.minimax(depth-1, False)
        tree[col] = subtree
        max_eval = max(max_eval, eval)
        self.undo_drop(col, row)
    return max_eval, tree
```

```
else:
    min_eval = float('inf')
    for col in range(self.width):
        row = self.simulate_drop(col, 2)
        if row is None: # dont calculate for invalid moves
            continue
        eval , subtree = self.minimax(depth-1, True)
        tree[col] = subtree
        min_eval = min(min_eval, eval)
        self.undo_drop(col, row)
    return min_eval, tree
```

Alpha-Beta Pruning

Alpha-Beta Pruning is an enhancement of Minimax that eliminates portions of the game tree which cannot affect the final decision. It keeps two

parameters—α (best already found for the maximizer) and β (best already found for the minimizer)—and prunes any branch where β ≤ α.
This optimization yields the same decision outcome as plain Minimax but runs significantly faster, expanding far fewer nodes.

```
for col in range(self.width):
        # "!!!!!!!!!
        row = self.simulate_drop(col, 1)
        if row is None: # dont calculate for invalid moves in the SIMULATION
            continue
        eval_val , subtree = self.alpha_beta(depth - 1, alpha, beta, 1 ^ maximizing_player)
        # bst_eval = max(bst_eval, eval) if maximizing_player else min(bst_eval, eval)
        best_tree[col] = subtree
        self.undo_drop(col, row)
        if maximizing_player:
            bst_eval = max(bst_eval, eval_val)

            alpha = max(alpha, eval_val)
        else:

            bst_eval = min(bst_eval, eval_val)
            beta = min(beta, eval_val)
        if alpha >= beta:
            break
```

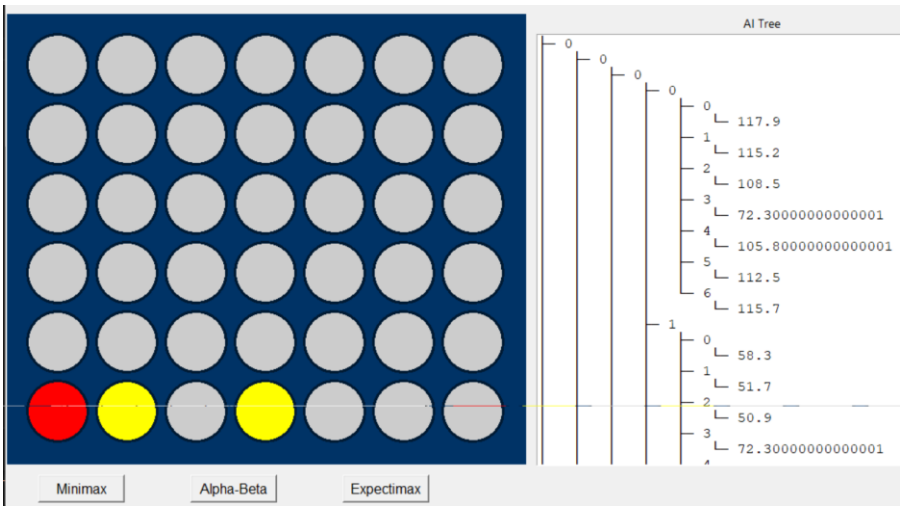
Expected Minimax (Expectimax)

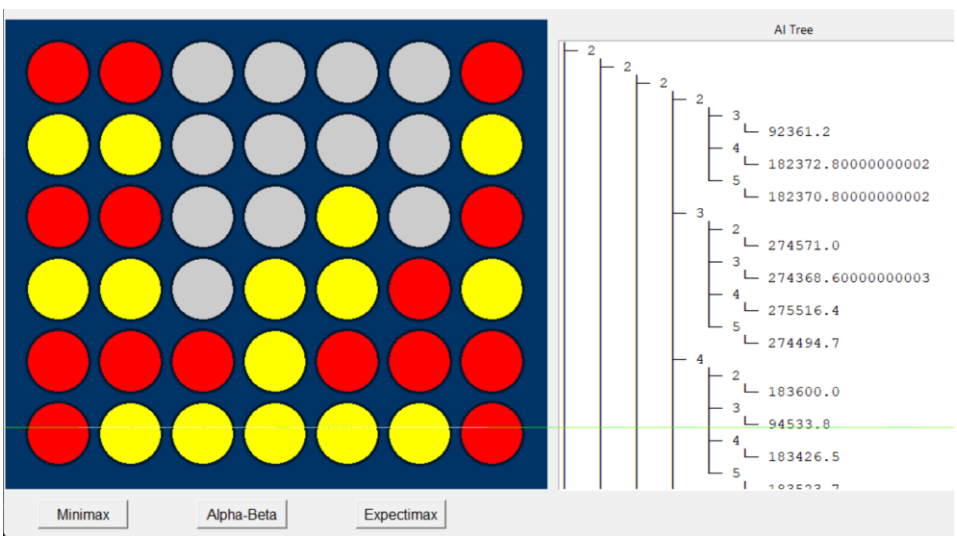
Expected Minimax, or Expectimax, extends the Minimax model by treating the opponent's moves as **probabilistic** rather than strictly adversarial. Each move is evaluated as the weighted average of its possible outcomes instead of the minimum.
In the intended version, the AI assumes a 0.6 probability that the opponent's piece falls in the selected column, and 0.2 for each adjacent column (or 0.4 if only one neighbor exists). This results in more "human-like" behavior where the opponent is not assumed to play perfectly.

```
    expectation = 0
    for p, c in zip(probs, cols):
        row = self.simulate_drop(c,2)
        if row is None:
            continue
        eval_val, subtree = self.exex_minimax(depth - 1, True)

        if c==col: # store tree only for main col
            tree[col] = subtree
        expectation += p*eval_val
        self.undo_drop(col, row)

    total += expectation
    count +=1

if count == 0:
    # no valid moves --> terminal node
    val = self.calc_heuristic(1)
    return val, val

return total/count, tree
```

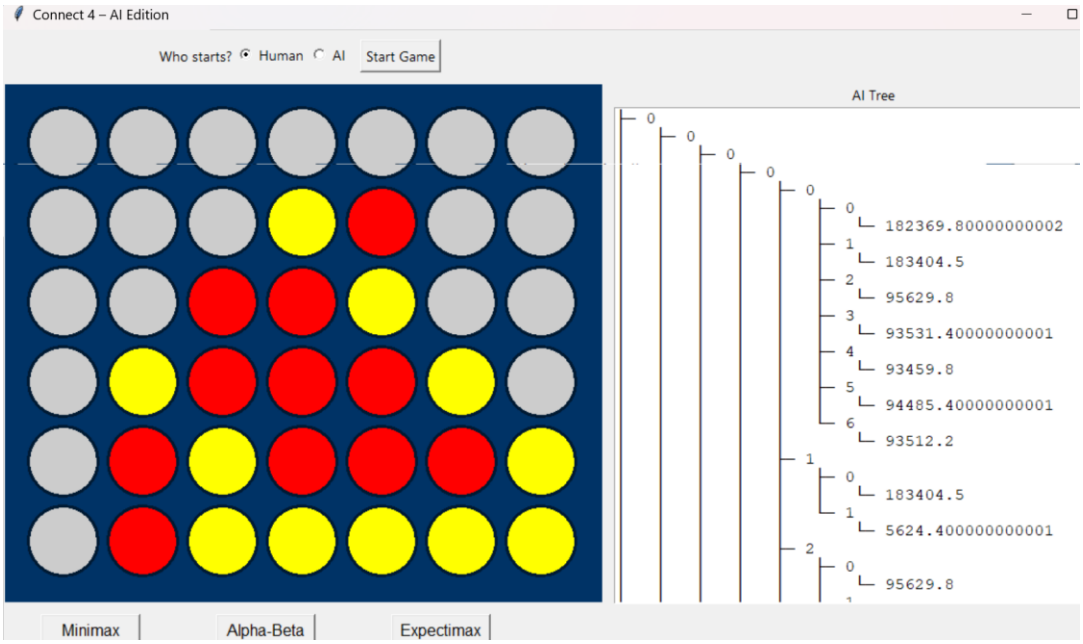
4. Sample Runs

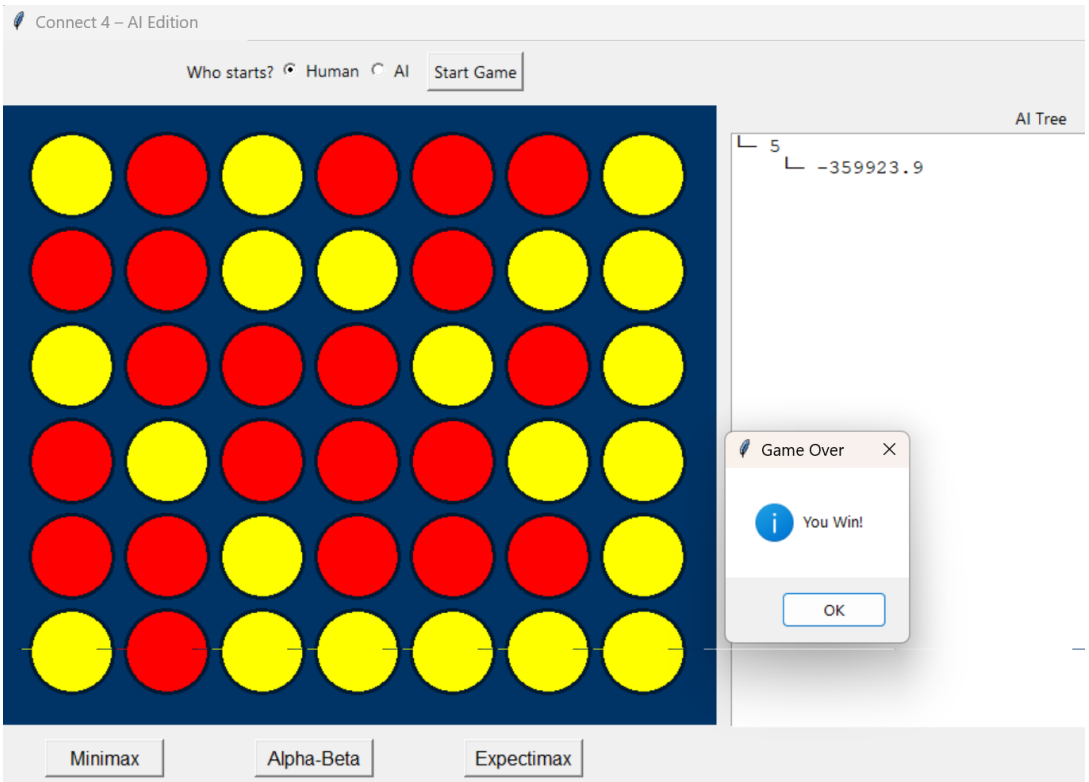
Minimax:





Alpha-beta pruning:





5. Comparison

```
=== Depth K = 2 ===
Minimax:     best_col=0,  time=0.002s,  nodes=56
Alpha-Beta:  best_col=3,  time=0.002s,  nodes=56
Expectimax:  best_col=6,  time=0.006s,  nodes=140

=== Depth K = 3 ===
Minimax:     best_col=0,  time=0.014s,  nodes=399
Alpha-Beta:  best_col=3,  time=0.009s,  nodes=264
Expectimax:  best_col=6,  time=0.044s,  nodes=1064

=== Depth K = 4 ===
Minimax:     best_col=0,  time=0.105s,  nodes=2800
Alpha-Beta:  best_col=3,  time=0.048s,  nodes=1315
Expectimax:  best_col=5,  time=0.045s,  nodes=1166

=== Depth K = 5 ===
Minimax:     best_col=0,  time=0.738s,  nodes=19607
Alpha-Beta:  best_col=3,  time=0.159s,  nodes=4707
Expectimax:  best_col=5,  time=0.200s,  nodes=5779
```

6. Data structures used

- grid: 2D list of integers (0 = empty, 1 = AI, 2 = Human).
- tree: Nested dictionary {col: subtree} used for visualization.